

Algoritmi e Strutture Dati – 09/02/2023 – Parte A

Esercizio -1 Iscriverti allo scritto entro la scadenza. In caso di inadempienza, -1 al voto finale.

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su tutti i fogli consegnati. Consegnare foglio A4 e foglio protocollo di bella. In caso di inadempienza, -1 al voto finale.

A1 – Complessità – Punti ≥ 8

Si consideri questo algoritmo:

```

int P(int[] A, int n)
{
    return R(A, 1, n)
}

int R(int[] A, int start, int end)
{
    if start == end then
        return A[start]
    int tot = 0
    int i = start
    int j = end
    while i ≤ j and A[i] == A[j] do
        tot = tot + A[i] + A[j]
        i = i + 1
        j = j - 1
    int mid = ⌊(end + start)/2⌋
    tot = tot + R(A, start, mid)
    tot = tot + R(A, mid + 1, end)
    return tot
}

```

Scrivere l'equazione di ricorrenza associata a questo algoritmo e calcolare la complessità computazionale che ne deriva, facendo particolare attenzione al caso pessimo e al caso ottimo.

A2 – Broadcast – Punti ≥ 10

Si supponga che si debba inviare un messaggio a tutti i nodi di un albero binario. All'inizio, solo la radice conosce il messaggio. Ad ogni turno, ogni nodo che conosce il messaggio può inviarlo ad uno solo dei suoi figli (all'altro dovrà inviarlo al turno successivo); al termine del turno, i nodi ricevono il messaggio e possono spedirlo il turno successivo.

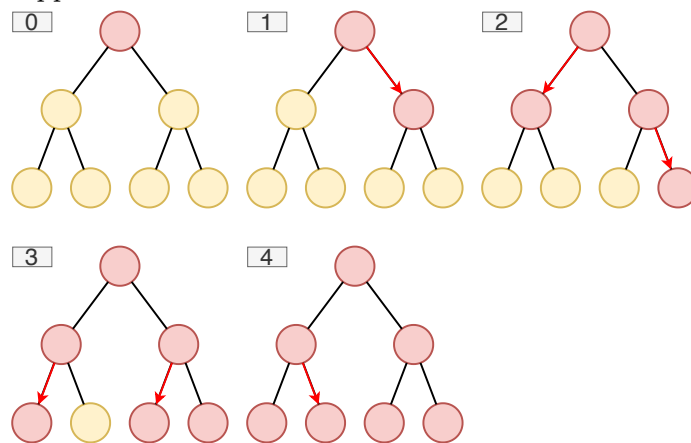
Scrivere un algoritmo

int minRounds(TREE T)

che restituisce il numero minimo di turni necessari affinché tutti i nodi di un albero binario T non vuoto vengano a conoscenza del messaggio.

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale.

Per esempio, nell'albero perfetto seguente, il numero di round necessari per raggiungere tutti i nodi è pari a 4. In rosa i nodi che hanno ricevuto il messaggio, in ocra quelli che non l'hanno ancora ricevuto; le frecce rosse indicano il messaggio che viene spedito durante un round; per ogni figura marcata i , la situazione illustrata è quella al termine del round i -esimo, tranne $i = 0$ che rappresenta la situazione iniziale.



A3 – Minimo locale – Punti ≥ 12

All'interno di un vettore A contenente $n \geq 3$ interi distinti, un minimo locale è un elemento $A[i]$ tale che $A[i-1] > A[i] < A[i+1]$, con $1 < i < n$.

Scrivere un algoritmo

int findLocalMinimum(**int**[] A, **int** n)

che prenda in input un vettore A contenente n interi per cui vale la relazione: $A[1] > A[2]$ e $A[n-1] < A[n]$, e restituisca l'indice di un minimo locale in tempo $O(\log n)$. Algoritmi con complessità lineare o superiore non verranno considerati.

Si può dimostrare che se i primi due e gli ultimi due elementi di un vettore sono tali che $A[1] > A[2]$ e $A[n-1] < A[n]$, allora A contiene un minimo locale.

Dimostrare la correttezza dell'algoritmo proposto.

Per esempio, se $A = [17, 15, \mathbf{13}, 14, \mathbf{12}, 18]$, ci sono due minimi locali, 13 e 12. L'algoritmo può restituire 3 o 5 (indici che partono da 1).